COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010 – Data Structure	**SECTION:**	CPE21S2
DATE PERFORMED:	August 06, 2026	**DATE SUBMITTED:**	August 07, 2026
NAME:	Macatangay Justin Jake B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1	**[Queue]**

1. PROCEDURE OUTPUT

ILO A

Screenshot

ILOA > main.cpp > display(std::queue<char>)

```cpp
#include <iostream>
#include<queue>

void display(std::queue<char> copyQ);
int main(){

    // create an object:
    std::queue<char> myQ;

    //use the enqueue operation
    myQ.push('J');
    myQ.push('A');
    myQ.push('K');
    myQ.push('E');
    std::cout<<"current front is: "<< myQ.front()<<std::endl;
    std::cout<<"current back is: "<< myQ.back()<<std::endl;
    display(myQ);

    //use the dequeue operation
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();

    //check the queue if it is empty
    std::cout<<"is the queue empty" << myQ.empty() <<std::endl;
  return 0;
}

//note only use the member functions of the queue STL
void display(std::queue<char> copyQ){
    //create a copy of the queue
    std::queue<char> temp = copyQ;
    // loop until empty
    while(!temp.empty()){


        //display the front
        std::cout<<" " <<temp.front() <<" ";
        //dequeue the front
        temp.pop();
    }
    // add a new line
    std::cout<<std::endl;
}
```

Output

```
current front is: J
current back is: E
 J  A  K  E
 A  K  E
 K  E
 E
is the queue empty1
```

ILO B

Screenshot

```cpp
#include <iostream>
#include "queueLL.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    if(isEmpty(&front, &back)){
        std::cout << "The queue is empty\n";
    }

    enqueue('J', &front, &back);
    std::cout<<"front: "<< front->data <<" "<<"back: " <<back->data<<std::endl;
    enqueue('A', &front, &back);
    std::cout<<"front: "<< front->data <<" "<<"back: " <<back->data<<std::endl;
    enqueue('K', &front, &back);
    std::cout<<"front: "<< front->data <<" "<<"back: " <<back->data<<std::endl;
    enqueue('E', &front, &back);
    std::cout<<"front: "<< front->data <<" "<<"back: " <<back->data<<std::endl;

    //testing the all display
    std::cout<<"Queue elements: ";
    displayQ(&front, &back);

    std::cout<<"Testing the dequeue operator: \n ";
    dequeue(&front, &back);
    std::cout<<"front: "<< front->data <<" "<<"back: " <<back->data<<std::endl;
    dequeue(&front, &back);
    std::cout<<"front: "<< front->data <<" "<<"back: " <<back->data<<std::endl;
    dequeue(&front, &back);
    dequeue(&front, &back);
    dequeue(&front, &back);


    return 0;
}
```

Ouput

```
The queue is empty
front: J back: J
front: J back: A
front: J back: K
front: J back: E
Queue elements: J
Testing the dequeue operator:
```

queueLL.h

ILOB > C queueLL.h > displayQ<T>(qNode<T>**, qNode<T>**)

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H


template <typename T>
class qNode{
    public:
       T data;
       qNode* next;
};

template <typename T>
qNode<T>* new_node(T newData){

    //
    qNode<T>* newNode = new qNode<T>;

    //store the newData to the newNode
    newNode -> data = newData;
    //point the newNode next to null
    newNode -> next = nullptr;

    return newNode;


}

//enqueue
template <typename T>
void enqueue(T newData, qNode<T>**frontPtr, qNode<T>** backPtr ){
    //create a new node
    qNode <T>* newNode = new_node(newData);

    //check if the queue is empty
    if((*frontPtr)==nullptr && (*backPtr)== nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
        return;
    }

    //if not then insert at the back
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
```

```
45  }
46  //dequeue
47  template <typename T>
48  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
49      //
50      if((*frontPtr) == nullptr && (*backPtr) == nullptr){
51          std::cout<< "The queue is empty"<<std::endl;
52          return;
53      }
54      //create a temporary variables to store the node to be deleted
55      qNode<T>* deleteTemp = nullptr;
56
57      //assign the current front to the deleteTemp
58      deleteTemp = (*frontPtr);
59
60      //chech if the queue has only 1 item
61      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
62          //means only 1 item is inside the queue
63          (*frontPtr) = nullptr;
64          (*backPtr) = nullptr;
65          delete deleteTemp;
66          return;
67      }
68
69      (*frontPtr) = (*frontPtr)->next;
70      deleteTemp->next = nullptr;
71
72      //delete node
73      delete deleteTemp;
74  }
75
```

```
76  //display
77  template <typename T>
78  void displayQ(qNode<T>** frontPtr, qNode<T>** backPtr){
79      //check is empty
80      if((*frontPtr) == nullptr && (*backPtr) == nullptr){
81          std::cout<< "The queue is empty"<<std::endl;
82          return;
83      }
84      //display all items
85      while ((*frontPtr) != nullptr){
86          std::cout<<(*frontPtr)->data;
87          if((*frontPtr)->next != nullptr){
88              std::cout<<" ";
89
90          }
91          (*frontPtr) = (*backPtr)->next;
92      }
93      std::cout<<std::endl;
94
95
96  };
97  //isEmpty
98  template <typename T>
99  bool isEmpty(qNode<T>** frontPtr,qNode<T>** backPtr){
100     return((*frontPtr) == nullptr && (*backPtr) == nullptr);
101 }
102
103
104
105
106 #endif
```

queueArr.h

Supplementary > C queueArr.h > ...

```h
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>

//ARRAY BASED QUEUE IMPLEMENTATION
template <typename T>
class QueueArr{
    private:
    //dynamic array allocate the queue
    T* q_array;

    //maximum number of elements in the queue
    size_t q_capacity;
    //current number of elements in the queue
    size_t q_size;
    //index of the front element in the queue
    int q_front;
    //index of the back element in the queue
    int q_back;


    public:
    //constructor
    QueueArr(size_t capacity = 10);
    //copy constructor
    QueueArr(const QueueArr<T>& other);
    //copy assignment operator
    QueueArr<T>& operator=(const QueueArr<T>& other);
    //destructor
    ~QueueArr();
    //queue operator
    bool Empty();
    bool Full();

    size_t Size();

    void clear();
    T Front();
    T Back();
    void Enqueue(T value);
    T Dequeue();
};
```

```cpp
// Constructor
template <typename T>
QueueArr<T>::QueueArr(size_t capacity) {
    q_capacity = capacity;
    q_array = new T[q_capacity];
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

// Copy Constructor
template <typename T>
QueueArr<T>::QueueArr(const QueueArr<T>& other) {
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;
    q_array = new T[q_capacity];
    for (size_t i = 0; i < q_capacity; i++) {
        q_array[i] = other.q_array[i];
    }
}

// Copy Assignment Operator
template <typename T>
QueueArr<T>& QueueArr<T>::operator=(const QueueArr<T>& other) {
    if (this != &other) {
        delete[] q_array;
        q_capacity = other.q_capacity;
        q_size = other.q_size;
        q_front = other.q_front;
        q_back = other.q_back;
        q_array = new T[q_capacity];
        for (size_t i = 0; i < q_capacity; i++) {
            q_array[i] = other.q_array[i];
        }
    }
    return *this;
}

// Destructor
template <typename T>
QueueArr<T>::~QueueArr() {
    delete[] q_array;
}

//empty
```

```cpp
//empty
template <typename T>
bool QueueArr<T>::Empty(){
    //check if the size if zero
    return q_size == 0;
}

//full
template <typename T>
bool QueueArr<T>::Full(){
    //check size if its max capacity
    return q_size == q_capacity;
}

//size_t
template <typename T>
size_t QueueArr<T>::Size(){
    return q_size;
}

// Front
template <typename T>
T QueueArr<T>::Front(){
    if(Empty()){
        std::cout<<"Queue is Empty\n";
        return T();
    }
    return q_array[q_front];
}

// Back
template<typename T>
T QueueArr<T>::Back(){
    //Check if the queue is empty
    if(Empty()){
        std::cout<<"Queue is empty"<<std::endl;
        return T();
    }
    //return to the back
    return q_array[q_back];
}

```

```cpp
//clear all
template <typename T>
void QueueArr<T>::clear(){
    //reset the size and front and back index
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

//Void Enqueue
template <typename T>
void QueueArr<T>::Enqueue(T value){

    //check if the queue is full
    if(Full()){
        std::cout<<"Queue is full\n";
        return;
    }
    //move q_back circularly
    q_back = (q_back + 1) % q_capacity;

    //store  the value to the back
    q_array[q_back] = value;

    //increment the q_size
    q_size++;
}
```

```
160
161  //T Dequeue
162  template <typename T>
163  T QueueArr<T>::Dequeue(){
164      //check if the queue is empty
165      if(Empty()){
166          std::cout<<"Queue is empty\n";
167          return T();
168      }
169      //store the front value to return
170      T value = q_array[q_front];
171
172      //move q_front circularly
173      q_front = (q_front + 1) % q_capacity;
174
175      //decrement the size
176      q_size--;
177
178      // reset indexes if queue becomes empty
179      if(Empty()) clear();
180
181      //return to temporary value
182      return value;
183  }
184
185  #endif
```

main.cpp of the queueArr.h

Screenshot

Supplementary > main.cpp > main()

```cpp
#include <iostream>
#include "queueArr.h"

int main() {
    // creating an object
    QueueArr<int> q(5);

    // enqueue 10, 20, 30, 40, 50:
    q.Enqueue(1);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.Enqueue(2);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.Enqueue(3);
    q.Enqueue(4);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.Enqueue(5);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.Enqueue(6); // Will display "Queue is full"

    std::cout << "\nremoved: " << q.Dequeue() << std::endl;
    std::cout << "removed: " << q.Dequeue() << std::endl;

    q.clear();

    std::cout << "is the queue empty? " << (q.Empty() ? "yes" : "no") << std::endl;

    return 0;
}
```

Output

```
Front: 1
Back: 1
Size: 1
Front: 1
Back: 2
Size: 2
Front: 1
Back: 4
Size: 4
Front: 1
Back: 5
Size: 5
Queue is full

removed: 1
removed: 2
is the queue empty? yes
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Screenshot

Supplementary > Supplementary.cpp > main()

```cpp
#include <iostream>
#include <string>

class Job {
private:
    int jobID;
    std:: string userName;
    int numberOfPages;

public:
    Job() : jobID(0), userName(""), numberOfPages(0) {}

    Job(int id, std::string name, int pages)
        : jobID(id), userName(name), numberOfPages(pages) {}

    int getJobID() const { return jobID; }
    std::string getUserName() const { return userName; }
    int getNumberOfPages() const { return numberOfPages; }

    void printDetails() const {
        std::cout <<"[Job ID: " << jobID
                  <<" | User: " << userName
                  <<" | Pages: " << numberOfPages << "]";
    }

};

struct Node {
    Job data;
    Node* next;
    Node(Job j) : data(j), next (nullptr) {}
};

class Printer {
    private:
    Node* frontPtr;
    Node* backPtr;
    int jobCount;

public:
    Printer() : frontPtr(nullptr), backPtr(nullptr), jobCount(0) {}

    ~Printer() {
        while (!isEmpty()) {
            processNextJob();
        }
    }

```

```
49      bool isEmpty() const {
50          return frontPtr == nullptr;
51      }
52
53  void addJob(const Job& newJob) {
54      Node* newNode = new Node(newJob);
55      if (isEmpty()) {
56          frontPtr = newNode;
57          backPtr = newNode;
58      } else {
59          backPtr->next = newNode;
60          backPtr = newNode;
61      }
62      jobCount++;
63      std::cout << "-> Added to Spooler: ";
64      newJob.printDetails();
65      std::cout << "\n";
66  }
67
68  void processNextJob() {
69      if(isEmpty()) {
70          std::cout << "Printer status: No pending jobs to process \n";
71          return;
72      }
73      Node* tempPtr = frontPtr;
74      std::cout << "=> Processing: ";
75      tempPtr->data.printDetails();
76      std::cout << " ... Done!\n";
77
78      frontPtr = frontPtr->next;
79      if (frontPtr == nullptr) {
80          backPtr = nullptr;
81      }
82
83      delete tempPtr;
84      jobCount--;
85  }
86
87  void displayPendingJobs() const {
88      if(isEmpty()) {
89          std::cout << "Printer Queue Status: Empty \n";
90          return;
91      }
```

```
    std::cout << "\n--- Current Pending Jobs in Printer Queue (" << jobCount<< ") ---\n";
    Node* current = frontPtr;
    int position = 1;
    while (current != nullptr) {
        std::cout << position++ << ". ";
        current->data.printDetails();
        std::cout << "\n";
        current = current->next;
    }
    std::cout << "----------------------------------------\n\n";
}
};

int main() {
    Printer officePrinter;

    std::cout << "=== OFFICE SHARED PRINTER ===\n\n";
    officePrinter.addJob(Job(101, "Justin", 19));
    officePrinter.addJob(Job(102, "Zamond", 20));
    officePrinter.addJob(Job(103, "Romuel", 21));

    officePrinter.displayPendingJobs();

    std::cout << "--- Printer begins processing ---\n";
    officePrinter.processNextJob();
    officePrinter.processNextJob();


    officePrinter.processNextJob();

    std::cout << "--- New Job sumbitted while printing ---\n";
    while (!officePrinter.isEmpty()) {
        officePrinter.processNextJob();
    }

    officePrinter.displayPendingJobs();

    return 0;


}
```

Output

```
=== OFFICE SHARED PRINTER ===

-> Added to Spooler: [Job ID: 101 | User: Justin | Pages: 19]
-> Added to Spooler: [Job ID: 102 | User: Zamond | Pages: 20]
-> Added to Spooler: [Job ID: 103 | User: Romuel | Pages: 21]

--- Current Pending Jobs in Printer Queue (3) ---
1. [Job ID: 101 | User: Justin | Pages: 19]
2. [Job ID: 102 | User: Zamond | Pages: 20]
3. [Job ID: 103 | User: Romuel | Pages: 21]
----------------------------------------

--- Printer begins processing ---
=> Processing: [Job ID: 101 | User: Justin | Pages: 19] ... Done!
=> Processing: [Job ID: 102 | User: Zamond | Pages: 20] ... Done!
=> Processing: [Job ID: 103 | User: Romuel | Pages: 21] ... Done!
--- New Job sumbitted while printing ---
Printer Queue Status: Empty
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

Conclusion

In this activity, I learned how queues work using both linked list and array implementations. I was able to perform the basic queue operations such as enqueue, dequeue, peek, and checking if the queue is empty. Through this exercise, I gained a better understanding of the FIFO (First In, First Out) principle and the differences between the two implementations. Overall, the activity improved my programming and problem-solving skills in implementing queue data structures in C++.